TME 6-7 : SYNCHRONISATIONS PAR SEMAPHORES

REALISATION D'UN CALCUL SYNCHRONISE

On veut paralléliser un calcul sur une variable partagée *result*. On considère :

- NADD processus additionneurs exécutant chacun la fonction *add_process(id)* pour *id* allant de 0 à NADD-1. *add_process(id)* appelle NB_TOURS fois la fonction *calcul_add(id)* puis ajoute le résultat à *result*.
- NMULT processus multiplicateurs exécutant chacun la fonction *mult_process(id)* pour *id* allant de 0 à NMULT-1. *mult_process(id)* appelle NB_TOURS fois la fonction *calcul_mult()* puis multiplie le résultat à la variable *result*.

Les synchronisations entre les processus sont réalisées au moyen de sémaphores et variables partagées en utilisant la bibliothèque `libsem` dont le descriptif est donné en annexe.

Les étapes pour le TME :

1) Copiez l'archive libsem.tgz disponible dans moodle dans votre répertoire.
2) Décompressez l'archive :
```
$ tar xvfz libsem.tgz
```
3) Compilez la bibliothèque :
```
$ cd libsem/src
$ make
```
4) Allez dans le répertoire du TME
```
$ cd ../tmeAddMult
$ make
```

Dans le répertoire du TME il y a trois squelettes de programme *add_mult_1.c*, *addmult_N.c* et *addmultdiv_N.c* à compléter.

1. MISE EN ŒUVRE AVEC UNE SEULE VARIABLE PARTAGEE

On considère d'abord une seule variable *result* définie dans le segment de mémoire partagée. Dans cette partie vous modifierez le programme **addmutl_1.c**.

1.1.

Modifiez le programme en fixant NADD à 100 et NMULT à 0. Compilez avec le makefile et lancez l'exécution plusieurs fois.

Que constatez-vous ? Comment expliquer ce phénomène ?

1.2.

Modifier le code de addmult_1.c pour que le résultat des 100 additions soit toujours le même (sans modifier la boucle ou les opérations de calcul dans add_process) ?

On considère maintenant NADD égal à 5 et NMULT égal à 5. On veut assurer un résultat déterministe même si les additions et multiplications ne sont pas commutatives. Pour cela à chaque tour de boucle on veut que les NADD processus *add_process* s'exécutent puis les NMULT processus *mult_process*.

1.3

Définissez les sémaphores nécessaires pour programmer cette synchronisation. Pour chaque sémaphore, vous préciserez son rôle et sa valeur initiale.

Indication : vous aurez besoin en plus de la variable partagée *result*, d'un compteur *count* (déjà défini dans le segment de mémoire partagée).

1.4

Programmez cette synchronisation.

1.3.

Testez votre programme avec différentes valeurs de NADD (>0) et NMUTL (>0) et NB_TOURS. Si l'affichage final indique en vert OK, le calcul final est correct. Si l'affichage est NOK en rouge, le calcul n'est pas bon, il faut revoir votre synchronisation.

2. MISE EN ŒUVRE AVEC PLUSIEURS VARIABLES

On considère maintenant NB_VAR variables *result* et le fichier **addmult_N.c**. Les calculs s'effectuent maintenant sur un tableau *result[NB_VAR]* déjà défini dans le segment de mémoire partagée.

Pour chaque variable *result[i]* il faut au final NB_TOURS fois NADD additions et NMULT multiplications.

Les calculs doivent être le plus parallèle possible : pendant les NMULT multiplications sur *result[i]*, les NADD additions peuvent se faire sur la variable suivante,
result[(i+1) % NB_VAR].

2.1.

Définissez les sémaphores et variables nécessaires pour programmer cette synchronisation. Pour chaque sémaphore et variable, vous préciserez son rôle et sa valeur initiale.

2.2.

Programmez cette synchronisation en utilisant la bibliothèque libsem.

2.3.

Testez votre programme avec différentes valeurs de NADD (>0) et NMUTL (>0) et NB_TOURS. Si l'affichage final indique en vert OK, le calcul final est correct. Si l'affichage est NOK en rouge, le calcul n'est pas bon, il faut revoir votre synchronisation.

3. PLUSIEURS VARIABLES PARTAGEES ET 1 DIVISEUR

Dans cette partie, vous modifierez le programme **addmutldiv_N.c**.

On souhaite maintenant à la fin de chaque tour réveiller un processus diviseur qui exécute la fonction *div_process()*. Ce processus divise par 3 le résultat produit par les processus multiplicateurs.

3.1.

Dans le fichier *addmultdiv_N.c*, copier le corps des fonctions *add_processes()* et *mult-processes()* qui vous avez programmé dans la section 2, puis modifiez les fonctions mult_processes et div_process pour implanter cette nouvelle synchronisation.

3.2.

Testez votre programme avec différentes valeurs de NADD (>0) et NMUTL (>0) et NB_TOURS.